

Analytics Drill-Down: Frontend Integration Guide

Hello Frontend Team! 🙌

The backend analytics-service is now ready for integration. To handle massive OLAP datasets (Apache Druid) without crashing the browser, we have implemented a **Stateless Lazy-Loading Architecture**.

The backend is completely blind to our business rules. Instead, **the frontend dictates the rules** by sending a hierarchy array in the request. The backend dynamically generates secure SQL, executes it, determines if a row is expandable, and returns a flat array.

Currently, the backend is running in **Mock Mode**, meaning it will instantly return realistic dummy financial data formatted exactly like the real database will. You can begin integrating and testing the UI immediately!

1. Endpoint Details

- **URL:** `http://localhost:8080/api/reports/drill-down` (*Adjust host/port as per your local/dev setup*)
- **Method:** POST
- **Headers:**
 - Content-Type: `application/json`
 - Authorization: `Bearer <JWT_TOKEN>` (*Required for future RBAC features*)

2. The Request Payload (What you send)

To power the dynamic engine, you must send **four** properties in the JSON body:

```
{
  "dimensions": ["Branch_name"],
  "metrics": ["jan", "feb", "mar"],
  "hierarchy": ["Circle_name", "Branch_name", "comp_code",
    "CGL_Number"],
  "filters": [
    {
      "field": "Circle_name",
      "operator": "EQUALS",
      "value": "North Circle"
    }
  ]
}
```

Payload Breakdown:

- **dimensions:** An array containing the single column you want to group by *right now*.

- **metrics:** An array of the number columns you want aggregated (summed).
- **hierarchy: (CRITICAL)** The full roadmap of the current report. The backend looks at this array to figure out if there is another level after the current dimension. If there is, it automatically returns "hasChildren": true.
- **filters:** An array of breadcrumbs. If you are loading the root table, this should be [] (empty). If a user clicks a branch, pass the circle and branch they clicked as filters.

3. The Response Payload (What you get)

The backend handles the deterministic ID generation and the hasChildren calculation. It returns a flat structure.

```
{
  "meta": {
    "status": "SUCCESS",
    "level": "Branch_name",
    "rowCount": 2,
    "executionTimeMs": 45
  },
  "data": [
    {
      "id": "branch_delhi_main",
      "name": "Delhi Main",
      "jan": 500000.00,
      "feb": 450000.00,
      "mar": 480000.00,
      "hasChildren": true
    },
    {
      "id": "branch_mumbai_south",
      "name": "Mumbai South",
      "jan": 300000.00,
      "feb": 320000.00,
      "mar": 310000.00,
      "hasChildren": false
    }
  ]
}
```

Key UI Triggers in Data:

- **id:** Use this as your React key={row.id}. The backend guarantees this is deterministic so your DOM won't flicker.
- **name:** The display label for that row.
- **hasChildren:** If true, render the [+] button. If false, hide it. You do not need to calculate this on the frontend anymore!

4. React Code Updates (Replacing Dummy Functions)

Here is how you can update the `fetchInitialData` and `fetchChildrenData` functions in your `ReportAnalysisScreen` component to connect to the real API.

```
// --- ACTUAL API LOGIC (Connecting to Spring Boot) ---

const API_URL = 'http://localhost:8080/api/reports/drill-down';

// 1. Fetch the Top-Level (Root) Data
const fetchInitialData = async (hierarchy, metrics = ["jan", "feb", "mar"]) => {
  const rootDimension = hierarchy[0];

  const jsonPayload = {
    dimensions: [rootDimension],
    metrics: metrics,
    hierarchy: hierarchy, // Send the full map!
    filters: [], // Empty for root
  };

  try {
    const response = await fetch(API_URL, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(jsonPayload),
    });

    if (!response.ok) throw new Error("Failed to fetch initial data");

    const result = await response.json();

    // Attach an empty nodeFilters array to root items so children
    have a starting point
    return result.data.map(row => ({
      ...row,
      level: 0, // Frontend tracking
      nodeFilters: [],
    }));
  } catch (error) {
    console.error("Error loading root data:", error);
    return [];
  }
};

// 2. Fetch the Children (Drill-Down) Data
const fetchChildrenData = async (parentNode, hierarchy, metrics =
```

```

["jan", "feb", "mar"]) => {
  const childLevel = parentNode.level + 1;
  const nextDimension = hierarchy[childLevel];

  // Build breadcrumbs based on where the user clicked
  const currentFilters = [
    ...(parentNode.nodeFilters || []),
    {
      field: hierarchy[parentNode.level], // e.g., 'Circle_name'
      operator: "EQUALS",
      value: parentNode.name,             // e.g., 'North Circle'
    },
  ];

  const jsonPayload = {
    dimensions: [nextDimension],
    metrics: metrics,
    hierarchy: hierarchy, // Always send the full map
    filters: currentFilters,
  };

  try {
    const response = await fetch(API_URL, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(jsonPayload),
    });

    if (!response.ok) throw new Error("Failed to fetch drill-down data");

    const result = await response.json();

    // Attach the updated filter trail to the children
    return result.data.map(row => ({
      ...row,
      level: childLevel, // Frontend tracking
      nodeFilters: currentFilters,
    }));
  } catch (error) {
    console.error("Error loading drill-down data:", error);
    throw error;
  }
};

```

Notes for UI State:

- You no longer need to calculate `hasChildren` based on the level. The backend provides it natively.
- You no longer need to generate IDs using `Math.random()`. Use the deterministic id returned from the API.
- Make sure to pass any required Authentication headers in the fetch calls once Security is enabled.